

Reviewer Pack — Hyperbolic Metric Entropy Correlation with SAT Clause Comple...

Entry 8d7fdaf23213 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 04:15:57 UTC

Hyperbolic Metric Entropy Correlation with SAT Clause Complexity

Entry ID: 8d7fdaf23213 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-31 04:15:57 UTC

1. Conjecture

Field A (mathematical branch): Geometric Group Theory **Field B** (complexity object): SAT (Satisfiability) Clause Complexity

Statement:

For every CNF φ with m clauses and n variables, the hyperbolic metric entropy of the associated Cayley graph $G(\varphi)$ on the free group F_n is lower-bounded by the resolution proof width $w(\varphi)$, such that $H(G(\varphi)) = \Omega(w(\varphi))$.

Rationale (proposer's reasoning):

Hyperbolic geometry has been previously used to study complexity measures, and the hyperbolic metric entropy of a group can be computed from its Cayley graph. The conjecture suggests a potential link between geometric properties of groups and the complexity of satisfiability problems, which could provide insights into the hardness of SAT.

Taxonomy category: GeometricGroupTheorySATClauseComplexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 1294bf11b0e9c197

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all CNFs φ with n variables and m clauses, $H(G(\varphi)) / w(\varphi) \geq 0.5$ for at least 24 out of 30 random seeds, where $H(G(\varphi))$ is the hyperbolic metric entropy of $G(\varphi)$ and $w(\varphi)$ is the resolution proof width.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.90	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - intitle:Hyperbolic Metric Entropy AND Geometric Group Theory AND SAT Clause Complexity - title:(Cayley graph OR free group) AND (hyperbolic metric entropy OR resolution proof width) AND SAT clause complexity - free group AND hyperbolic metric entropy AND lower bound ON resolution proof width IN context OF CNF

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    m, n = len(A), len(A[0])
    for i in range(m):
        max_row = i
        for j in range(i+1, m):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        for j in range(i+1, m):
            factor = A[j][i] / A[i][i]
            for k in range(n):
                A[j][k] -= factor * A[i][k]
    return A

def matrix_multiplication(A, B):
    m, n, p = len(A), len(B), len(B[0])
    C = [[0] * p for _ in range(m)]
    for i in range(m):
        for j in range(p):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def determinant(A):
    m, n = len(A), len(A[0])
    if m != n:
        raise ValueError("Matrix must be square")
    if n == 1:
```

```

        return A[0][0]
det = 0
for j in range(n):
    submatrix = [row[:j] + row[j+1:] for row in A[1:]]
    det += (-1) ** j * A[0][j] * determinant(submatrix)
return det

def resolution_width(cnf):
    clauses = list(cnf)
    n = len(clauses)
    width = 0
    for i in range(n):
        for j in range(i+1, n):
            if -clauses[i][0] in clauses[j]:
                width = max(width, len(set(clauses[i]) | set(clauses[j])))
    return width

def hyperbolic_metric_entropy(G):
    n = len(G)
    adjacency_matrix = [[0] * n for _ in range(n)]
    for u in range(n):
        for v in G[u]:
            adjacency_matrix[u][v] = 1
            adjacency_matrix[v][u] = 1
    laplacian_matrix = [[0] * n for _ in range(n)]
    degree_sum = sum(sum(row) for row in adjacency_matrix)
    for i in range(n):
        laplacian_matrix[i][i] = degree_sum - 2 * sum(adjacency_matrix[i])
        for j in range(i+1, n):
            laplacian_matrix[i][j] = -adjacency_matrix[i][j]
            laplacian_matrix[j][i] = -adjacency_matrix[j][i]
    eigenvalues = []
    for _ in range(n):
        largest_eigenvalue = float('-inf')
        v = [random.random() for _ in range(n)]
        v /= math.sqrt(sum(x**2 for x in v))
        for _ in range(100):
            Av = matrix_multiplication(laplacian_matrix, v)
            Av /= max(abs(x) for x in Av)
            v = Av
        largest_eigenvalue = max(largest_eigenvalue, abs(v[0]))
    eigenvalues.append(largest_eigenvalue)
    return sum(math.log(eig) / n for eig in eigenvalues)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []
    for n in n_values:
        instances_tested = 0
        total_metric_value = 0
        max_n = n
        conjecture_holds = True
        counterexample = ""
        for _ in range(5):
            m = random.randint(n, 2*n)

```

```

cnf = {i: set(random.sample(range(-n, n+1), random.randint(1, n))) for i in range(m)}
G = {i: [] for i in range(n)}
for clause in cnf.values():
    for literal in clause:
        if literal > 0:
            G[literal-1].append(literal-n)
            G[literal+n-1].append(-literal+1)
        else:
            G[-literal-1].append(-literal+n)
            G[-literal+n-1].append(literal+1)
metric_value = hyperbolic_metric_entropy(G)
width = resolution_width(cnf)
if width == 0:
    continue
instances_tested += 1
total_metric_value += metric_value / width
max_n = max(max_n, n)
if metric_value / width < 0.5:
    conjecture_holds = False
    counterexample = f"n={n}, m={m}"
results.append({
    "metric_name": "H(G( $\phi$ )) / w( $\phi$ )",
    "metric_value": total_metric_value / instances_tested,
    "instances_tested": instances_tested,
    "n_max": max_n,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
})
return {
    "seed": seed,
    "results": results
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result["results"])
    metric_values = [r[0]["metric_value"] for r in results for r in r]
    support_fraction = sum(1 for r in results for r in r if r[0]["conjecture_holds"]) / len(results)
    mean = sum(metric_values) / len(metric_values)
    std = math.sqrt(sum((x - mean)**2 for x in metric_values) / len(metric_values))
    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) for r in result if not r[0]["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"n={result[0]['counterexample']}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5665962e.py", line 140, in <module>
    result = run_trial(seed)
             ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5665962e.py", line 108, in run_trial
    G[literal+n-1].append(-literal+1)
    ~~~~~
KeyError: 6
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from verifying the conjecture's support condition. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13632
2	preregistration	ollama_remote	glm4:latest	0	0	10453
3	novelty	ollama_remote	glm4:latest	0	0	14190
4	novelty	ollama_remote	glm4:latest	0	0	8952
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16502
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17305
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20305
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22976
9	judge	ollama_remote	glm4:latest	0	0	10168

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 134483 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/8d7fdaf23213.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/8d7fdaf23213.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/8d7fdaf23213.tar.gz` (if generated)